Name: Fiona Yi Ting Wang(fionayiw)
Course :95702-B
Assignment Name: Project 4 Task2

Introduction:

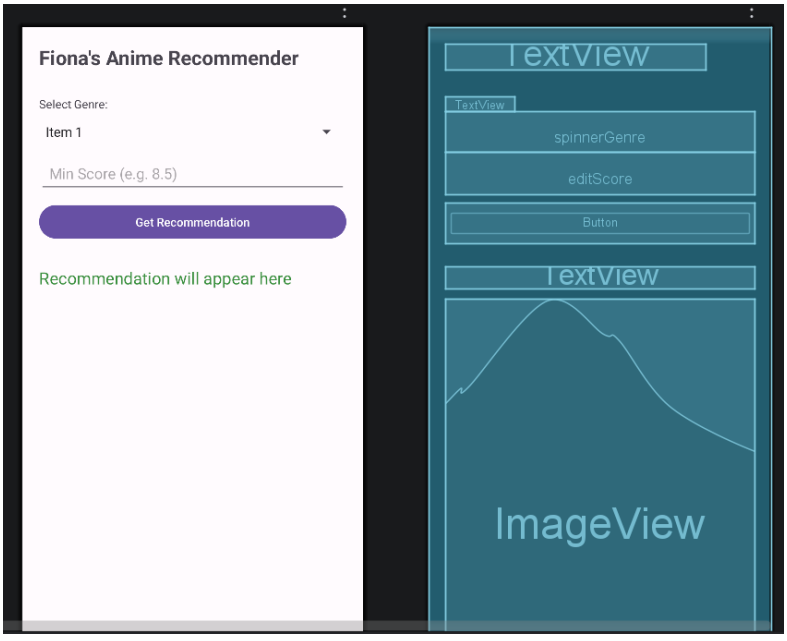
The web service integrates with the Jikan API (v4), an open-source REST API for MyAnimeList.

- Purpose: This API provides extensive anime metadata, including titles, scores, synopsis, and image URLs, which are essential for the recommendation logic
- Endpoint Used: https://api.jikan.moe/v4/anime

1. Android Application

The client-side application is a native Android app designed to provide anime recommend services.

- **1.a (Views):** The layout uses various Views, such as a Spinner for genre selection, an EditText for score input, and an ImageView for displaying posters.



- **1.b (Input):** Users provide input via the genre selection and minimum score field

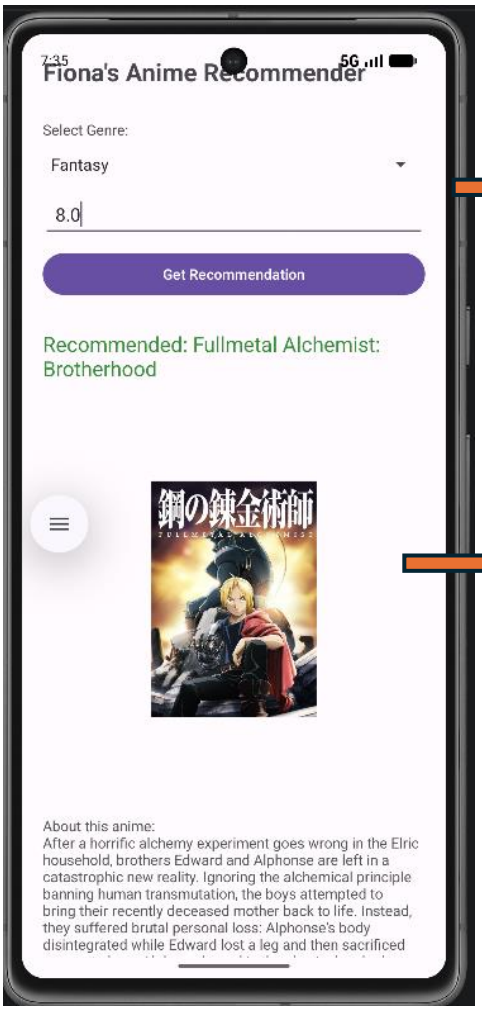
This code snippet from MainActivity.java demonstrates the fulfillment of the following requirements:

- **1.c (HTTP Request):** It shows the use of HttpURLConnection to open a connection and set the request method to GET.
- **1.d (Receive and Parse JSON):** It shows the extraction of the input stream and the use of JSONObject to parse the title, image_url, and synopsis from the web service's response.
- **1.e (Display Information):** It demonstrates how runOnUiThread() is used to update the UI elements (txtTitle, txtSynopsis) and how the Glide library is used to asynchronously load the anime poster into the ImageView.
- **Concurrency :** It clearly shows that network operations are performed on a background thread (new Thread(() -> { ... }).start()) to keep the application responsive, fulfilling core Android development principles.

Key Code Snippet (MainActivity.java-AnimeRecommendApp):

```
// Network operations must run on a background thread to avoid blocking the UI
new Thread(() -> {
    HttpURLConnection conn = null;
    try {
        // Construct the URL with multiple dynamic query parameters
        URL url = new URL(API_URL + "?genre=" + genreId + "&score=" + score);

        // Open HTTP connection and configure the request
        conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");

        // Set custom User-Agent for server-side logging (stored in MongoDB)
        conn.setRequestProperty("User-Agent", "Fiona-Android-App-Project4");

        // Check for HTTP OK response (200)
        if (conn.getResponseCode() == 200) {
            BufferedReader in = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
            StringBuilder resp = new StringBuilder();
            String line;
            while ((line = in.readLine()) != null) {
                resp.append(line);
            }

            // Parse the raw JSON string returned by the Servlet
            JSONObject json = new JSONObject(resp.toString());
            String title = json.getString("title");
            String imageUrl = json.getString("image_url");
            String synopsis = json.getString("synopsis");

            // Update UI elements back on the Main UI Thread
            runOnUiThread(() -> {
                txtTitle.setText("Recommended: " + title);
                txtSynopsis.setText("About this anime: \n" + synopsis);

                // Load the poster image asynchronously using the Glide library
                Glide.with(MainActivity.this)
                        .load(imageUrl)
                        .placeholder(android.R.drawable.ic_menu_gallery)
                        .into(imgAnime);
            });
```

```
            } else {
                final int status = conn.getResponseCode();
                runOnUiThread(() -> Toast.makeText(this, "Server error: " + status,
Toast.LENGTH_SHORT).show());
            }
        } catch (java.io.IOException e) {
            e.printStackTrace();
            runOnUiThread(() -> Toast.makeText(this, "Network Error: Check URL or
connectivity.", Toast.LENGTH_LONG).show());
        } catch (Exception e) {
            e.printStackTrace();
            runOnUiThread(() -> Toast.makeText(this, "App Error: Review logcat for
details.", Toast.LENGTH_LONG).show());
        } finally {
            if (conn != null) conn.disconnect();
        }
    }).start();
```



Users select a genre, input a minimum score, and click the 'Get Recommendation' button

Display random anime that fits the user input , it would fetch title,anime image and introduction

2. Web Service Implementation

Description: The Java Servlet (AnimeServlet.java) acts as a middleware between the mobile client and the Jikan API .

- **2.a & 2.b (API & Request):** A RESTful API is deployed to handle incoming mobile requests .
- **2.c (Business Logic):** The server fetches data from the Jikan 3rd-party API.

Key Code Snippet (AnimeServlet.java):

```
/* Construct a RESTful request to the Jikan API with specific search
and sorting parameters */
String searchUrl = "https://api.jikan.moe/v4/anime?genres=" + genreId +
"&min_score=" + minScore + "&order_by=score&sort=desc";
URL url = new URL(searchUrl);
HttpURLConnection conn = (HttpURLConnection) url.openConnection();
conn.setRequestMethod("GET");
```

- **2.d (Data Minimization):** To save bandwidth (Requirement 2.d), the Servlet filters the 3rd-party response and returns only 4 essential fields: title, score, image_url, and synopsis.

Key Code Snippet (AnimeServlet.java):

```
if (animeList.length() > 0) {
    /* Select a random entry to ensure a diverse user experience with every request */
    int randomIndex = new Random().nextInt(animeList.length());
    JSONObject picked = animeList.getJSONObject(randomIndex);

    /* Filter out unnecessary metadata to minimize network latency and mobile processing load */
    filteredResponse.put("title", picked.getString( key: "title"));
    filteredResponse.put("score", picked.getDouble( key: "score"));
    filteredResponse.put("image_url", picked.getJSONObject( key: "images").getJSONObject( key: "jpg").getString( key: "large_image_url"));
    filteredResponse.put("synopsis", picked.optString( key: "synopsis",  defaultValue: "No description available."));
}
```

3. Handle Error Conditions

The application is designed to handle failures gracefully without crashing:

- **Input Validation:** The Android app validates user input and alerts the user (via Toast) if the score field is empty.

```
/* Apply default values if the mobile app did not provide specific search criteria */
if (genreId == null) genreId = "1";
if (minScore == null) minScore = "8.0";
```

- **Exception Handling:** Both the Android client and Java Servlet utilize try-catch blocks to manage network timeouts, JSON parsing errors, and 3rd-party API unavailability.

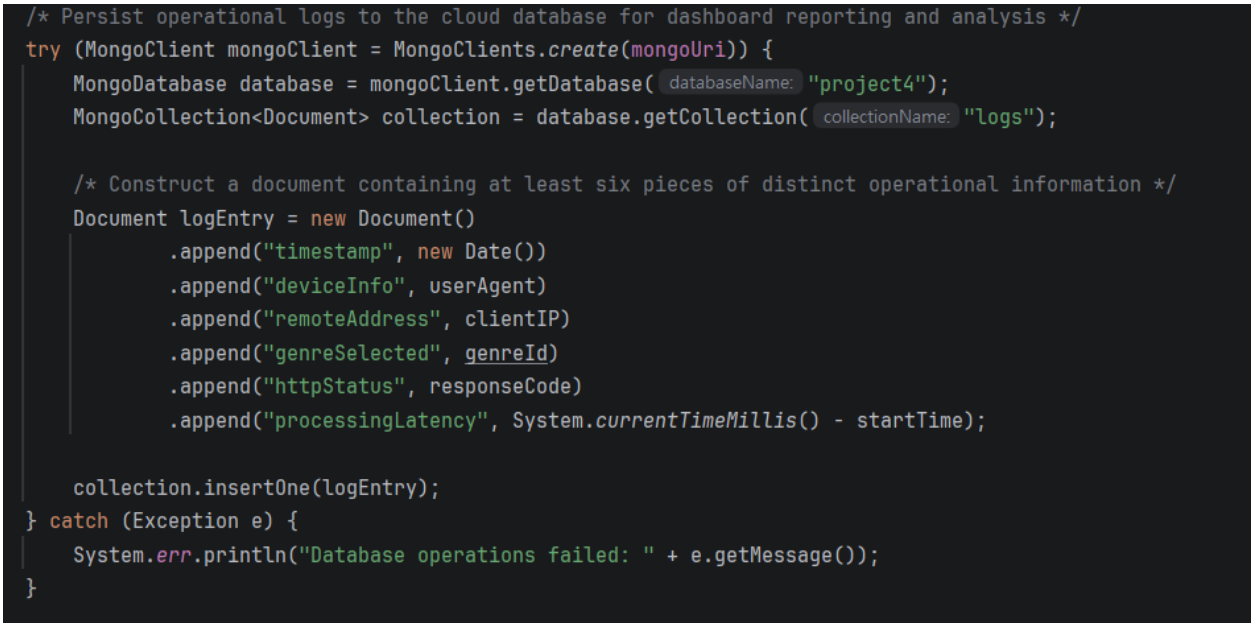
- **User Feedback:** If a network error occurs, a user-friendly error message is displayed, ensuring the app remains functional and stable.

4. Logging Useful Information

The system logs six distinct pieces of metadata for every interaction to facilitate operational oversight:

1. Request Timestamp: The exact time the request reached the server.
2. Device Information: Captured via the User-Agent header (e.g., *Fiona-Android-App-Project4*).
3. Client IP Address: The remote address of the mobile client.
4. Genre Selection: The specific Genre ID requested by the user.
5. API HTTP Status: The response code from the 3rd-party API (e.g., 200 OK).
6. Processing Latency: The time taken (in milliseconds) to process the request and log to the database.

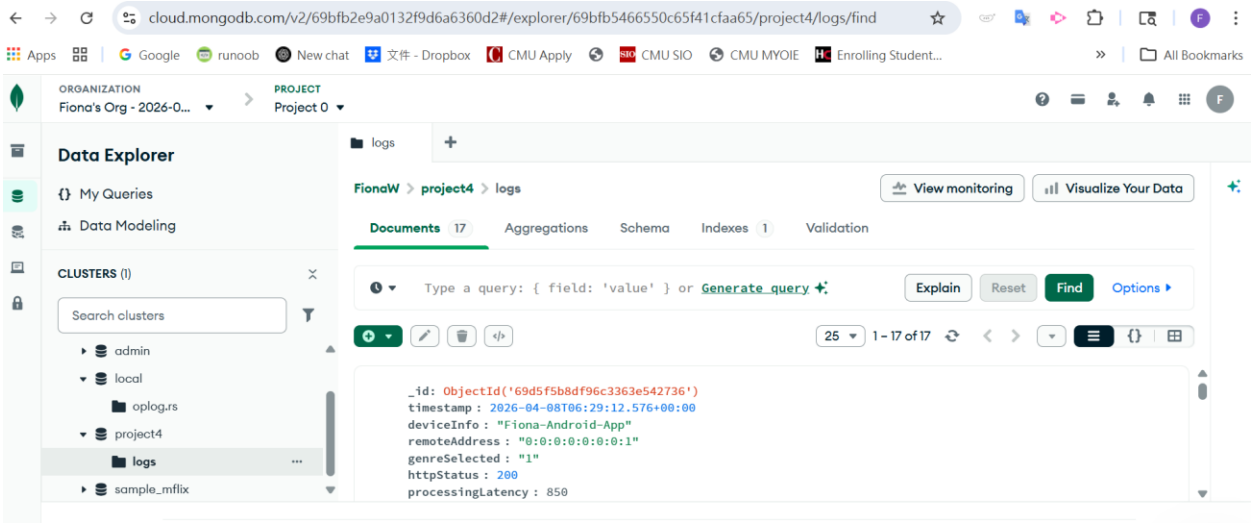
Key Code Snippet (AnimeServlet.java):



```
/* Persist operational logs to the cloud database for dashboard reporting and analysis */
try (MongoClient mongoClient = MongoClients.create(mongoUri)) {
    MongoDatabase database = mongoClient.getDatabase( databaseName: "project4");
    MongoCollection<Document> collection = database.getCollection( collectionName: "logs");

    /* Construct a document containing at least six pieces of distinct operational information */
    Document logEntry = new Document()
            .append("timestamp", new Date())
            .append("deviceInfo", userAgent)
            .append("remoteAddress", clientIP)
            .append("genreSelected", genreId)
            .append("httpStatus", responseCode)
            .append("processingLatency", System.currentTimeMillis() - startTime);

    collection.insertOne(logEntry);
} catch (Exception e) {
    System.err.println("Database operations failed: " + e.getMessage());
}
```

5. MongoDB Database Storage

All interaction logs are persistently stored in a MongoDB Atlas NoSQL database cluster in the cloud. The web service utilizes the MongoDB Java Driver to perform programmatic insert operations, ensuring that log data is preserved across server restarts and available for the dashboard.

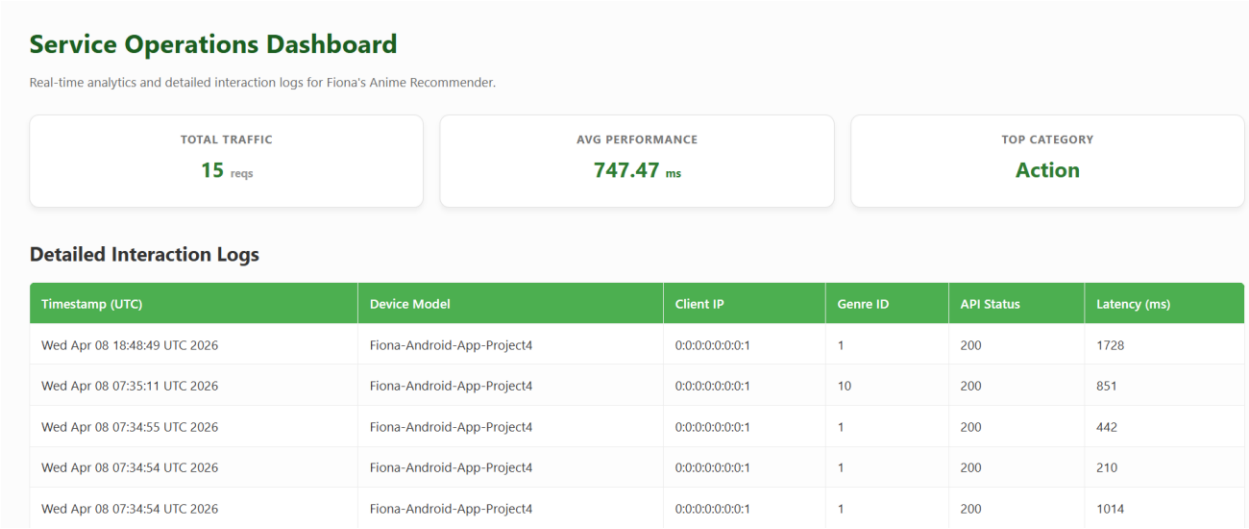


6. Operations Dashboard and Analytics

The web-based dashboard provides a comprehensive interface for system monitoring:

- 6.a. Unique URL: The dashboard is accessible via a unique, persistent cloud URL hosted on GitHub Codespaces: https://didactic-capybara-94x97q5jj96c79vq-8080.app.github.dev/
- 6.b. Operations Analytics: The dashboard calculates and displays three key metrics:
    1. Total Traffic: Total number of requests processed.
    2. Performance Metric: Average processing latency across all requests.
    3. User Preference Analysis: The most popular genre based on user selection frequency.
- 6.c. Formatted Logs: Detailed interaction logs are retrieved from the database and rendered in a **clean HTML table**. Each entry displays the operational metadata in a human-readable format, strictly avoiding raw JSON or XML to ensure a professional

user interface .



Service Operations Dashboard

Real-time analytics and detailed interaction logs for Fiona's Anime Recommender.

TOTAL TRAFFIC	AVG PERFORMANCE	TOP CATEGORY
15 reqs	747.47 ms	Action

Detailed Interaction Logs

Timestamp (UTC)	Device Model	Client IP	Genre ID	API Status	Latency (ms)
Wed Apr 08 18:48:49 UTC 2026	Fiona-Android-App-Project4	0:0:0:0:0:0:0:1	1	200	1728
Wed Apr 08 07:35:11 UTC 2026	Fiona-Android-App-Project4	0:0:0:0:0:0:0:1	10	200	851
Wed Apr 08 07:34:55 UTC 2026	Fiona-Android-App-Project4	0:0:0:0:0:0:0:1	1	200	442
Wed Apr 08 07:34:54 UTC 2026	Fiona-Android-App-Project4	0:0:0:0:0:0:0:1	1	200	210
Wed Apr 08 07:34:54 UTC 2026	Fiona-Android-App-Project4	0:0:0:0:0:0:0:1	1	200	1014

7. Deployment to GitHub Codespaces

The web service and dashboard are fully deployed on GitHub Codespaces within a Docker container. The port 8080 visibility is set to Public to allow the Android application to access the RESTful API and the dashboard via a unique cloud URL.

* **Dashboard URL**: https://didactic-capybara-94x97q5jj96c79vq-8080.app.github.dev/

* **Web Service (API) Endpoint**: https://didactic-capybara-94x97q5jj96c79vq-8080.app.github.dev/getAnime